Topic 5: Generators and Iterables

1. Yield Syntax

A function that contains `yield` is a *generator function*. The `yield` is similar to `return`, but we can re-enter the function after using the value. Generator functions return a generator object when called. The `next` function extracts the next yielded value from the generator object. If there are no more values to yield (i.e. the generator object is empty), then `next` raises `StopIteration`. Generator objects can be converted to lists using the `list` function and they can be looped over using for loops.

Problem 1. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 1
    yield 2
    yield 3
try:
    result = g()
    x = next(result)
    x = next(result)
    print('x=', x)
    result = g()
    print('list=', list(result))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 2. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def countdown(n):
    for i in range(n, 0, -1):
        yield i
try:
    print('list(countdown(5))=', list(countdown(5)))
    print('sum(countdown(5))=', sum(countdown(5)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 3. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 1
    yield 2
    yield 3
try:
    it = g()
    print('list(it)=', list(it))
    print('sum(it)=', sum(it))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 4. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 1
    yield 2
    yield 3
try:
    it = g()
    print('next(it)=', next(it))
    print('list(it)=', list(it))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 5. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def countdown(n):
    for i in range(n, 0, -1):
        yield i
try:
    print('list(countdown(3))=', list(countdown(3)))
    print('next(countdown(0))=', next(countdown(0)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

1.2 Infinite Generators

A generator which never raises `StopIteration` describes an infinite sequence. These infinite sequences cannot be represented as lists or other non-generator data structures. If you try to convert these infinite generators into a list, python will raise `MemoryError`.

Problem 6. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    it = nats()
    print('next(it)=', next(it))
    print('next(it)=', next(it))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 7. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    print('sum=', sum(nats()))
    print('list=', list(nats()))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 6 / 16 = 0.38

1.3 Generator Comprehensions

Generators can be constructed using the comprehension syntax just like lists. The advantage is that generators will consume $O(1)$ memory, but lists will consumer $O(n)$ memory. The disadvantage is that they can only be "consumed" once.

Problem 8. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [1, 2, 3, 4]
try:
    print('list(xx)=', list(x * x for x in xs))
    print('sum(xx)=', sum(x * x for x in xs))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 9. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [1, 2, 3, 4]
n = 2
gen = (n * x for x in xs)
n = 3
try:
    print('n now=', n)
    print('list(gen)=', list(gen))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 10. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [[1, 2], [3, 4], [5]]
try:
    print('list=', list(y for x in xs for y in x))
    print('sum=', sum(y for x in xs for y in x))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

2. Gnerators that Modify Generators

Many useful generators modify other generators (i.e. they take a generator as input).

2.1 Writing a generator

`take(g, n)` is a generator that pulls exactly `n` values from another generator `g`. Because both are generators, state (the position within `g`) is shared: consuming through `take` also advances `g`.
If `g` runs out before `n` items are taken, `next(g)` raises `StopIteration`, which propagates out of `take`.

Problem 11. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1

def take(g, n):
    for _ in range(n):
        yield next(g)
try:
    print('list(t)=', list(take(nats(), 5)))
    print('sum(t)=', sum(take(nats(), 4)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $15 \;/\; 16 = 0.94$

Problem 12. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1

def take(g, n):
    for _ in range(n - 1):
        yield next(g)
try:
    print('list(t)=', list(take(nats(), 5)))
    print('sum(t)=', sum(take(nats(), 4)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $10 \;/\; 16 = 0.62$

Problem 13. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1

def take(g, n):
    for _ in range(n):
        yield next(g)
try:
    g = nats()
    print('take=', list(take(g, 3)))
    print('next(g)=', next(g))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 14. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1

def take(g, n):
    for _ in range(n):
        yield next(g)
        next(g)
try:
    print('list(t)=', list(take(nats(), 4)))
    print('sum(t)=', sum(take(nats(), 3)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 15. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def small():
    yield 'a'
    yield 'b'

def take(g, n):
    for _ in range(n):
        yield next(g)
try:
    print('take2=', list(take(small(), 2)))
    print('take3=', list(take(small(), 3)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

Problem 16. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 1
    yield 2
    yield 3
try:
    it = g()
    print('list(it)=', list(it))
    print('sum(it)=', sum(it))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 0 / 0 = 1.00

Problem 17. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 1
    yield 2
    yield 3
try:
    it = g()
    print('next(it)=', next(it))
    print('list(it)=', list(it))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 0 / 0 = 1.00

Problem 18. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def countdown(n):
    for i in range(n, 0, -1):
        yield i
try:
    print('list(countdown(3))=', list(countdown(3)))
    print('next(countdown(0))=', next(countdown(0)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 0 / 0 = 1.00

2.2 Lazy Built-ins

`zip`, `map`, `filter`, and `enumerate` return lazy iterators, not lists. `zip` stops at the shortest input; `map` stops at the shortest input when given more than one iterable. All of them can be forced to a list with `list()`.
Subtlety: when a generator is passed to two arguments of the same lazy consumer (e.g. `zip(g, g)`), each argument pulls from the same underlying iterator, so the two output positions are interleaved.
`filter(None, xs)` keeps only truthy elements; `0`, `''`, `[]`, `None`, and `False` are all dropped.

Problem 19. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [1, 2, 3, 4]
ys = ['a', 'b', 'c']
try:
    print('zip(xs, ys)=', list(zip(xs, ys)))
    print('zip(ys, xs)=', list(zip(ys, xs)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 20. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 1
    yield 2
    yield 3
    yield 4
try:
    it1 = g()
    print('zip2=', list(zip(it1, it1)))
    it2 = g()
    print('zip3=', list(zip(it2, it2, it2)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 21. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def g():
    yield 'a'
    yield 'b'
    yield 'c'
try:
    it = g()
    print('enum=', list(enumerate(it)))
    print('again=', list(enumerate(it)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 22. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [1, 2, 3, 4]
ys = [10, 20, 30]
try:
    print('map=', list(map(lambda a, b: a + b, xs, ys)))
    print('map2=', list(map(lambda a, b: a * b, xs, ys)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 23. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
xs = [0, 1, '', 'a', [], [1], None, False]
try:
    print('filter=', list(filter(None, xs)))
    print('count=', len(list(filter(None, xs))))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 24. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    print('start')
    print('list=', list(map(lambda x: x * x, nats())))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

2.3 itertools

The `itertools` module provides efficient lazy building blocks.

- `count(start, step)` — infinite arithmetic sequence.
- `cycle(xs)` — repeats `xs` forever.
- `chain(a, b, ...)` — yields items from each iterable in turn.
- `islice(it, stop)` or `islice(it, start, stop, step)` — slices a lazy sequence like a list slice.

`count` and `cycle` are infinite; consume them with `islice`, or `list()` will raise `MemoryError`.

Problem 25. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    print('islice=', list(itertools.islice(itertools.count(0, 2), 5)))
    print('sum=', sum(itertools.islice(itertools.count(1, 3), 4)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 26. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    print('islice=', list(itertools.islice(itertools.count(1, 2), 5)))
    print('sum=', sum(itertools.islice(itertools.count(0, 3), 4)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 27. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    print('cycle=', list(itertools.islice(itertools.cycle([1, 2, 3]), 7)))
    print('sum=', sum(itertools.islice(itertools.cycle([4, 5]), 5)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 28. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    it = itertools.chain([1, 2, 3], [4, 5, 6])
    print('first=', list(itertools.islice(it, 4)))
    print('rest=', list(it))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 29. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    print('start')
    print('list=', list(itertools.cycle([1, 2, 3])))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

2.4 Pipelines

Individual lazy tools compose into larger lazy pipelines. Each stage pulls from the stage before it, and `islice` or `take` terminates an otherwise infinite chain.
Common shapes: `islice(map(f, xs), n)`, `sum(islice(chain(a, b), n))`, `take(filter(p, xs), n)`. The same principles apply no matter how many stages the pipeline has.

Problem 30. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
try:
    print('sum=', sum(itertools.islice(map(lambda x: x * x, fib()), 5)))
    print('list=', list(itertools.islice(map(lambda x: 2 * x, fib()), 5)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 9 / 16 = 0.56

Problem 31. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
def fib():
    a, b = 0, 1
    while True:
        yield a
        a, b = b, a + b
try:
    print('sum=', sum(itertools.islice(map(lambda x: x * x, fib()), 4)))
    print('list=', list(itertools.islice(map(lambda x: 2 * x, fib()), 6)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

Problem 32. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
try:
    print('a=', list(itertools.islice(itertools.chain([1, 2, 3], itertools.count(0)), 6)))
    print('b=', sum(itertools.islice(itertools.chain([1, 2, 3], itertools.count(0)), 5)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 13 / 16 = 0.81

Problem 33. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
import itertools
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    print('evens=', list(itertools.islice(filter(lambda x: x % 2 == 0, nats()), 5)))
    print('threes=', list(itertools.islice(filter(lambda x: x % 3 == 0, nats()), 5)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 34. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def take(g, n):
    for _ in range(n):
        yield next(g)
try:
    print('a=', list(take(map(lambda x: x * x, [1, 2, 3, 4]), 2)))
    print('b=', sum(take(map(lambda x: x + 1, [1, 2, 3, 4]), 3)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 35. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def nats():
    i = 0
    while True:
        yield i
        i += 1
try:
    print('start')
    print('list=', list(filter(lambda x: True, nats())))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 11 / 16 = 0.69

3. Desugaring and Dunders

Python for loops are syntactic sugar over the iter/next functions. For example the for loop below

```
for x in xs:
    foo()
```

gets converted into

```
it = iter(xs)
try:
    while True:
        x = next(it)
        foo()
except StopIteration:
    pass
```

We can loop over the `list`, `range`, etc types because they implement the generator protocol. See PEP255 `https://peps.python.org/pep-0255/` for details.
Any python object that implements the `__iter__` dunder is called an *iterable*.

Problem 36. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1
try:
    sum = 0
    for x in Countdown(3):
        sum += x
    print('sum=', sum)
    print('list=', list(Countdown(3)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 10 / 16 = 0.62

Problem 37. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i
try:
    print('list=', list(Countdown(3)))
    print('sum=', sum(Countdown(3)))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 38. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class CountdownIter:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1

class Countdown:
    def __init__(self, n):
        self.n = n
    def __iter__(self):
        return CountdownIter(self.n)
try:
    c = Countdown(3)
    print('a=', list(c), 'b=', list(c))
    print('c=', list(c), 'sum=', sum(c))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 39. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1
try:
    c = Countdown(3)
    print('first=', list(c))
    print('second=', list(c))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: $14 \;/\; 16 = 0.88$

Problem 40. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1
try:
    c = Countdown(3)
    it = iter(c)
    print('next=', next(it))
    print('rest=', list(it))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

Problem 41. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
def countdown(n):
    for i in range(n, 0, -1):
        yield i

class CountdownIter:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1

class Countdown:
    def __init__(self, n):
        self.n = n
    def __iter__(self):
        return CountdownIter(self.n)
try:
    c1 = countdown(3)
    print('iter(c1) is c1:', iter(c1) is c1)
    c2 = Countdown(3)
    print('iter(c2) is c2:', iter(c2) is c2)
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 16 / 16 = 1.00

Problem 42. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class CountdownIter:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1

class Countdown:
    def __init__(self, n):
        self.n = n
    def __iter__(self):
        return CountdownIter(self.n)
try:
    c = Countdown(3)
    print('sum1=', sum(c))
    print('sum2=', sum(c))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 14 / 16 = 0.88

Problem 43. Write the output of the final command in the following shell script.

```
$ cd; rm -rf quiz; mkdir quiz; cd quiz
$ cat > foo.py <<EOF
class Countdown:
    def __init__(self, n):
        self.i = n
    def __iter__(self):
        return self
    def __next__(self):
        if self.i <= 0:
            raise StopIteration
        self.i -= 1
        return self.i + 1
try:
    c = Countdown(3)
    print('for=', [x for x in c])
    print('next after for=', next(c))
except TypeError:
    print('TypeError')
except StopIteration:
    print('StopIteration')
except MemoryError:
    print('MemoryError')
EOF
$ python3 foo.py
```

Fraction of LLMs with correct answer: 15 / 16 = 0.94

LLM Model Performance

